

CodeCrafter — AI Coding Agent

Version: 2.1.0 · **Model:** Llama 3.3 70B (via Groq) · **Languages:** Python, C++, JavaScript, HTML/CSS

A local command-line AI coding agent that reads, writes, executes, and debugs code in a sandboxed workspace. Built for learning, rapid prototyping, and development assistance.

Author: Muhammad Rameez — rameezalipacific@gmail.com

Table of Contents

- 1. [Features](#)
- 2. [Prerequisites & Installation](#)
- 3. [Quick Start](#)
- 4. [Session Management](#)
- 5. [CLI Commands](#)
- 6. [Project Architecture](#)
- 7. [Available Tools](#)
- 8. [Security & Safety](#)
- 9. [Error Handling & API Management](#)
- 10. [Troubleshooting](#)
- 11. [Best Practices](#)
- 12. [Future Improvements](#)
- 13. [Contributing & Support](#)

Features

Multi-Language Code Execution

Language	Runtime	Features
Python	<code>python</code>	30s timeout, full stdout/stderr capture
C++	<code>g++</code>	Compile + execute, custom compiler flags
JavaScript	Node.js / Bun	ES6+ support, runtime selection
HTML/CSS	Default browser	Auto-preview, file:// URL launch
Shell	System shell	Any terminal command via <code>run_command</code>

Session Management (v2.1.0)

Sessions are stored in a dedicated `sessions/` directory (outside workspace) with automatic timestamped naming:

- **Persistent history** — Conversations survive app restarts
- **Multiple sessions** — Create, list, load, and delete sessions
- **Session visibility** — See session name in the banner and prompt
- **Corruption recovery** — Malformed sessions are backed up and gracefully recovered

Security Hardening (v2.1.0)

- **Path traversal protection** — All file operations use `os.sep`-aware boundary checks
- **Command blacklist** — 15+ regex patterns block destructive commands (`rm -rf /` , `format C:` , fork bombs, credential theft, etc.)
- **Workspace sandbox** — All file operations restricted to `workspace/` directory

Intelligent Workspace Organization

- **Project requests** (e.g. "build a calculator") → dedicated folder created automatically
- **Simple scripts** (e.g. "write a fibonacci function") → created at workspace root
- **Auto-metadata** — `project_description.json` tracks file purposes and project context

Professional CLI Interface

- **Nerd Font icons** with ANSI color coding
- **Animated spinner** with random status words while waiting for AI
- **Token usage display** after every response (prompt + completion + total)
- **Verbose mode** (`--verbose`) for debugging with full function call details
- **Personalized greeting** by name

API Key Rotation

- Load multiple Groq API keys from `api_keys.json`
 - Automatic silent rotation on rate limit (429) errors
 - No downtime — the user never sees a rate limit unless all keys are exhausted
-

Prerequisites & Installation

Requirements

Requirement	Details
Python	3.11+
Package Manager	<code>uv</code> (recommended) or <code>pip</code>
Groq API Key(s)	Free from console.groq.com
C++ Compiler	<code>g++</code> (optional, for C++ support)
Node.js or Bun	(optional, for JavaScript support)

Installation

```
# Clone the repo
git clone <your-repo-url>
cd CodeCrafter

# Install dependencies (using uv - recommended)
uv sync

# Or using pip
pip install groq
```

Configure API Keys

Create an `api_keys.json` file in the project root containing an array of Groq API keys:

```
[
  "gsk_your_first_api_key_here",
  "gsk_your_second_api_key_here"
]
```

Note: Even a single key works. Multiple keys enable automatic rotation when rate limits are hit.

Getting your key:

1. Visit [Groq Console](#)
2. Click "API Keys" → "Create API Key"
3. Copy and paste into `api_keys.json`

Quick Start

Run the Agent

```
# Standard mode
uv run main.py

# With verbose debugging
uv run main.py --verbose
```

First Launch

```

❏ CodeCrafter v2.1.0
┌───────────────────────────────────────────────────────────────────────────────────┐
❏ Model: llama-3.3-70b-versatile | Mode: Interactive
❏ Session: session_2026-02-27_18-40-00
❏ Type help for commands, exit to close
❏ Hello Rameez, ready to build something solid

```

Example Interaction

[session_2026-02-27_18-40-00] Rameez > Create a snake game in HTML with retro graphics

- ✓ Wrote retro_snake_game.html (3,743 chars)
- ✓ Opening retro_snake_game.html

CodeCrafter

A retro-themed snake game has been created. The game features a black background, green snake, and red food pellets. Use arrow keys to play.

1,234 in | 456 out | 1,690 total

Session Management

Sessions are named automatically with timestamps (e.g. `session_2026-02-27_18-40-00`) and stored in the `sessions/` directory at the project root — **outside** the workspace, so they never pollute your project files.

How It Works

Behavior	Details
Auto-save	Session saved after every AI response and on exit
Auto-load	Most recent session is loaded on startup
Max messages	Last 30 messages kept per session (configurable in <code>config.py</code>)
Corruption recovery	Broken JSON files are renamed <code>.corrupt</code> and a fresh session starts

Session Commands

Command	Action
<code>sessions</code>	List all saved sessions with dates and message counts
<code>session new</code>	Save current session, start a fresh one
<code>session load <name></code>	Switch to a different session (saves current first)
<code>session delete <name></code>	Delete a session (can't delete the active one)
<code>clear</code>	Clear the current session's message history

Example

```
[session_2026-02-27_18-40-00] Rameez > sessions
```

```
 Saved Sessions
```

```
→ session_2026-02-27_18-40-00 2026-02-27 18:40 | 12 msgs
   session_2026-02-26_14-30-00 2026-02-26 14:30 | 8 msgs
   session_2026-02-25_09-15-00 2026-02-25 09:15 | 22 msgs
```

```
[session_2026-02-27_18-40-00] Rameez > session new
```

```
New session: session_2026-02-27_19-05-23
```

CLI Commands

Command	Description
help	Show all available commands
sessions	List saved sessions
session new	Start a new session
session load <name>	Load a specific session
session delete <name>	Delete a session
clear	Clear current session history
exit / quit / e / q	Save session and exit

Any other input is sent to the AI as a prompt.

Project Architecture

```
CodeCrafter/
├─ main.py                # Core agent loop, CLI, session management
├─ config.py              # Global configuration (paths, limits)
├─ api_keys.json          # Groq API keys (gitignored)
├─ pyproject.toml         # Dependencies (groq)
├─ README.md              # This file
├─ functions/             # Tool implementations with OpenAI-format schemas
│   ├── get_files_info.py  # List files with metadata (size, modified date)
│   ├── get_file_content.py # Read file contents (with truncation)
│   ├── write_file.py      # Create/modify files safely
│   ├── delete_file.py     # Remove files safely
│   ├── run_python_file.py # Execute Python scripts
│   ├── run_cpp_file.py    # Compile & run C++ programs
│   ├── run_js_file.py     # Execute JavaScript (Node/Bun)
│   ├── preview_html_file.py # Open files in default app/browser
│   ├── run_command.py     # Execute shell commands (with blocklist)
│   └─ ...                 # Project description utilities
├─ sessions/              # Session storage (gitignored)
│   ├── session_2026-02-27_18-40-00.json
│   └─ ...
└─ workspace/             # Sandboxed working directory
    └─ [your files & projects]
```

Configuration (config.py)

```
PROJECT_FOLDER_NAME = "workspace" # Sandboxed working directory
SESSIONS_DIR = "sessions/"        # Session file storage
MAX_SESSION_MESSAGES = 30         # Messages kept per session
MAX_FILE_CHARS = 30000            # Max chars read from a file
AUTO_UPDATE_DESCRIPTION = True    # Auto-track project metadata
```

Available Tools

The AI agent has access to these tools, each defined with an OpenAI-compatible function schema:

File Operations

Tool	Description	Security
get_files_info	Recursively list files with size & modified date	Path-bounded
get_file_content	Read file contents (truncated at 10K chars)	Path-bounded
write_file	Create/overwrite files; auto-creates directories	Path-bounded
delete_file	Remove a file safely	Path-bounded

Code Execution

Tool	Description	Timeout
<code>run_python_file</code>	Execute <code>.py</code> files with optional args	30s
<code>run_cpp_file</code>	Compile with <code>g++</code> and execute; cleanup binary	30s
<code>run_js_file</code>	Execute with Node.js or Bun	30s
<code>preview_file</code>	Open any file in its default app (HTML → browser)	N/A
<code>run_command</code>	Execute shell commands (with dangerous command blocking)	30s

Security & Safety

Sandbox Boundaries

All file operations are restricted to `WORKING_DIR` (default: `workspace/`):

```
# These are blocked by path traversal detection:
"../../etc/passwd"      # Directory traversal → blocked
"/etc/shadow"          # Absolute path outside workspace → blocked
"../api_keys.json"      # Escaping workspace → blocked
```

The path check uses `os.sep`-aware comparison (not naive `startswith`) to prevent edge cases like a directory named `workspace-evil` matching `workspace`.

Command Blocklist

The `run_command` tool blocks dangerous patterns before execution:

Category	Blocked Patterns
Filesystem destruction	<code>rm -rf /</code> , <code>format C:</code> , <code>del /s /q C:\</code> , <code>mkfs</code> , <code>dd if=</code> , fork bombs
Data exfiltration	<code>curl -d @file</code> , <code>wget --post-file</code>
System manipulation	<code>shutdown</code> , <code>reboot</code> , <code>reg delete HKLM</code> , <code>chmod 777 /</code>
Credential theft	<code>cat ~/.ssh/*</code> , <code>type ..\credentials</code> , reading <code>shadow / passwd</code>

Important: `run_command` still uses `shell=True` for flexibility. The blocklist reduces risk but cannot prevent all possible misuse. Always review AI-suggested commands.

API Key Security

- Keys are loaded from `api_keys.json` (gitignored)
- Never hardcode keys in source files
- Rotate keys if you suspect exposure

Best Security Practices

1. Run in an isolated directory — don't point `WORKING_DIR` at important files
2. Review generated code before executing it
3. Use `--verbose` mode to see every tool call
4. Keep `api_keys.json` in `.gitignore` (it is by default)
5. This tool is for **development only** — not for production use

Error Handling & API Management

Automatic API Key Rotation

When a rate limit (429) is hit, the agent silently rotates to the next key in `api_keys.json` . The user only sees an error if **all keys** are exhausted.

Error Messages

Error	What Happens
Rate limit (429)	Silent key rotation; error shown only if all keys exhausted
Invalid request (400)	Key rotation attempted, then context trimmed, then error shown
Service unavailable (503)	Friendly retry message
Context too large	Older messages trimmed automatically
Max steps (25)	Warning shown, conversation reset

Token Usage

Token usage is displayed after every response:

🗨 1,234 in | 456 out | 1,690 total

In `--verbose` mode, token usage is shown after every step (including intermediate tool calls).

Troubleshooting

“No API keys found”

Cause: `api_keys.json` is missing or empty.

Fix: Create `api_keys.json` with at least one valid Groq key:

```
["gsk_your_key_here"]
```

“ModuleNotFoundError: No module named 'groq'”

Fix:

```
uv sync  
# or  
pip install groq
```

“Rate limit reached”

Cause: All API keys in `api_keys.json` have been rate-limited.

Fix: Wait a moment, or add more keys to `api_keys.json`.

“Command blocked for safety”

Cause: The AI tried to run a command matching the dangerous command blocklist.

Fix: This is intentional. If you need to run the command, do it manually in your terminal.

“Timeout after 30 seconds”

Cause: Code execution exceeded the time limit.

Fix: Optimize the code, or modify the timeout in the relevant function file.

Strange characters in terminal

Cause: Terminal doesn't support Nerd Font symbols.

Fix: Install a [Nerd Font](#) (e.g. JetBrainsMono Nerd Font) or modify the `Icons` class in `main.py`.

Session corrupted

Cause: Session JSON file was malformed (e.g. manual editing or crash during write).

Fix: Automatic — the app renames the file to `.corrupt` and starts fresh. Your old data is preserved as `<session_name>.json.corrupt` in `sessions/`.

Best Practices

Write Clear Prompts

```
# Good – specific and clear
Create a Python function that takes a list of numbers and returns
the sum of all even numbers. Include error handling for invalid input.

# Bad – vague
Write some code
```

Break Down Complex Tasks

```
# Step 1
Create a basic calculator class with add, subtract, multiply, divide

# Step 2
Add error handling for division by zero

# Step 3
Create a test file to verify all operations
```

Use Verbose Mode for Debugging

```
uv run main.py --verbose
```

Shows exact function calls, arguments, results, and per-step token usage. Invaluable when the AI isn't doing what you expect.

Organize with Sessions

- Use `session new` when switching to a different task
- Use `sessions` to find and reload old conversations
- Session context helps the AI understand your ongoing work

Future Improvements

- **Git integration** — Auto-commits, diff analysis, branch management
 - **Streaming responses** — Real-time token-by-token output
 - **Image/multimodal support** — Analyze screenshots, diagrams, UI mockups
 - **Plugin system** — Custom tool registration for third-party integrations
 - **Automated testing** — Pytest integration with coverage reporting
 - **Docker support** — Containerized execution for safer sandboxing
-

Contributing & Support

Reporting Issues

Create an issue with:

- Clear description of the problem
- Steps to reproduce
- Expected vs. actual behavior
- Error messages or terminal output
- Environment (Python version, OS, terminal)

Contributing Code

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/your-feature`)
3. Make changes with clear commit messages
4. Test thoroughly
5. Submit a Pull Request

Contact

Email: rameezalipacific@gmail.com

License

MIT License — free to use, modify, and distribute with attribution.

Last Updated: February 27, 2026 · **Version:** 2.1.0 · **Author:** Muhammad Rameez